

## Lecture 3: Asymptotic Notations

- Why asymptotic notations for algorithm efficiency?
- Big  $O$ , Big  $\Theta$ , and Big  $\Omega$
- Properties of asymptotic notations

School of Computing, Queen's University

### Algorithm efficiency analysis

- Characterize an algorithm's efficiency
  - to compare the relative performance of alternative algorithms
  - to identify the bottleneck of an algorithm
- When the input size increases without bound (asymptotic!)
  - inputs can be large in reality
  - asymptotically more efficient algorithm will be a better choice (except for very small inputs)

## Order of running time growth

- The lower-order terms and coefficients become insignificant
- The order of growth of the running time of an algorithm

$$T_1(n) = 3n^3 - n^2 + 2 \qquad \Theta(n^3)$$

$$T_2(n) = 50n^2 + 10 \qquad \Theta(n^2)$$

3

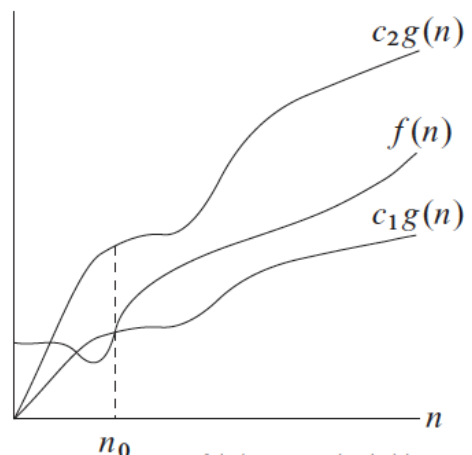
## Asymptotic notation $\Theta$

### • $\Theta$ -notation

$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that}$   
 $0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_0\}.$

$$f(n) \in \Theta(g(n)) \text{ or } f(n) = \Theta(g(n))$$

asymptotically tight bound



4

## Asymptotic notation $\Theta$

- $\Theta$ -notation

$$\frac{1}{2}n^2 - 3n = \Theta(n^2)$$

$$c_1n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2n^2$$

$$c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$$

5

## Asymptotic notation $\Theta$

- $\Theta$ -notation

$$6n^3 \neq \Theta(n^2)$$

$$6n^3 \leq c_2n^2$$

$$n \leq c_2/6$$

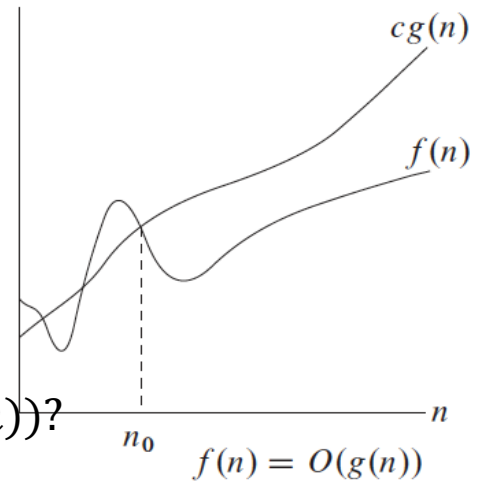
6

## Asymptotic notation $O$

- $O$ -notation

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$   
 $0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}.$

upper bound



If  $f(n) = \Theta(g(n))$ , does it imply  $f(n) = O(g(n))$ ?

7

## Asymptotic notation $O$

- $O$ -notation

$$\Theta(g(n)) \subseteq O(g(n))$$

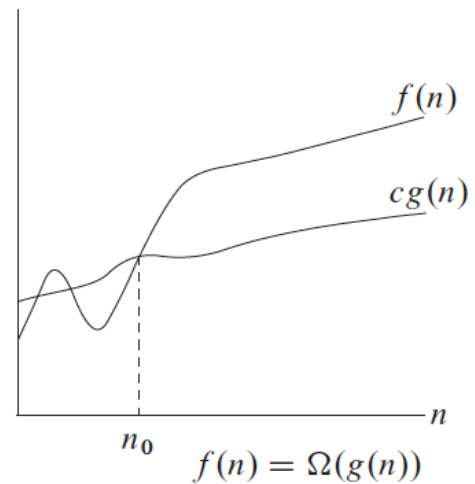
- If  $O$ -notation bounds the worst-case running time of an algorithm, it bounds the running time of the algorithm on every input.

# Asymptotic notation $\Omega$

- $\Omega$ -notation

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}$ .

lower bound



9

## Theorem

For any two functions  $f(n)$  and  $g(n)$ , we have  $f(n) = \Theta(g(n))$  if and only if  $f(n) = O(g(n))$  and  $f(n) = \Omega(g(n))$ . ■

$$an^2 + bn + c = \Theta(n^2)$$

$$an^2 + bn + c = O(n^2) \quad an^2 + bn + c = \Omega(n^2)$$

Are these equations correct?

1.  $n + 1 = \Theta(n)$
2.  $n + 1 = O(n)$
3.  $n + 1 = O(n^2)$
4.  $n + 1 = \Omega(n)$
5.  $n + 1 = \Omega(n^2)$
6.  $n + 1 = \Omega(1)$

Are these statements correct?

1. The running time of Insertion-sort is in  $\Omega(n)$
2. The running time of Insertion-sort is in  $O(n^2)$
3. The running time of Insertion-sort is in  $\Omega(n^2)$
4. The running time of Insertion-sort is in  $\Theta(n)$
5. The running time of Insertion-sort is in  $\Theta(n^2)$
6. The best-case running time of Insertion-sort is in  $\Theta(n)$
7. The worst-case running time of Insertion-sort is in  $\Theta(n^2)$

## Asymptotic notation in equations and inequalities

$$2n^2 + 3n + 1 = 2n^2 + \Theta(n)$$

$$2n^2 + 3n + 1 = 2n^2 + f(n) \quad \text{where}$$

$$f(n) \in \Theta(n) \quad \text{or} \quad f(n) = \Theta(n)$$

Using asymptotic notation in this manner can help eliminate inessential detail and clutter in an equation.

13

## Asymptotic notation in equations and inequalities

$$2n^2 + \Theta(n) = \Theta(n^2)$$

For any function  $f(n) \in \Theta(n)$ , there is some function  $g(n) \in \Theta(n^2)$  such that  $2n^2 + f(n) = g(n)$

$$\begin{aligned} 2n^2 + 3n + 1 &= 2n^2 + \Theta(n) \\ &= \Theta(n^2) \end{aligned}$$

14

- Transitivity

$f(n) = \Theta(g(n))$  and  $g(n) = \Theta(h(n))$  imply  $f(n) = \Theta(h(n))$

$f(n) = O(g(n))$  and  $g(n) = O(h(n))$  imply  $f(n) = O(h(n))$

$f(n) = \Omega(g(n))$  and  $g(n) = \Omega(h(n))$  imply  $f(n) = \Omega(h(n))$

- Reflexivity

$f(n) = \Theta(f(n))$ ,  $f(n) = O(f(n))$ , and  $f(n) = \Omega(f(n))$

- Symmetry

$f(n) = \Theta(g(n))$  if and only if  $g(n) = \Theta(f(n))$

$f(n) = O(g(n))$  if and only if  $g(n) = \Omega(f(n))$

15

## Reading

- Chapter 3: Characterizing Running Times

